

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

**«Белорусский государственный университет информатики и
радиоэлектроники»**

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по лабораторной работе №3

по дисциплине

«Генетические и эволюционные вычисления»

«Аппроксимация функции с помощью двухслойной нейронной сети»

Выполнил:

Лебедев Артем Владимирович
магистрант группы 556301

Преподаватель:

Нестеренков Сергей Николаевич
кандидат технических наук
доцент кафедры ПОИТ

Минск, 2026 г.

Содержание

1	Цель работы	2
2	Теоретические сведения	2
2.1	Двухслойная нейронная сеть	3
2.2	Предобработка и инициализация (как в newff)	3
2.3	Алгоритм обратного распространения ошибки	3
2.4	Ускоренные алгоритмы обучения	4
2.5	Теорема об универсальной аппроксимации	4
3	Ход работы	5
3.1	Исходные данные	6
3.2	Программная реализация	6
3.3	Обучение по заданию с мультипликацией	6
3.4	Серия экспериментов с разным числом нейронов	7
3.5	Доведение обучения до целевой ошибки	7
3.6	Влияние коэффициента обучения	9
3.7	Недообучение и переобучение	9
3.8	Интерактивное приложение	10
4	Результаты	10
5	Иллюстрации результатов	11
6	Выводы	19
7	Ответы на контрольные вопросы	20
8	Контрольные вопросы для Moodle	22

1 ЦЕЛЬ РАБОТЫ

Изучить архитектуру двухслойной нейронной сети (многослойного персептрона), принципы её обучения методом обратного распространения ошибки (Backpropagation), а также исследовать точность аппроксимации нелинейной функции в зависимости от количества нейронов во внутреннем (скрытом) слое.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Двухслойная нейронная сеть

Многослойный персептрон с одним скрытым и одним выходным слоем является универсальным аппроксиматором. Для скалярного входа t выход скрытого слоя из S нейронов вычисляется с нелинейной функцией активации «гиперболический тангенс» (tansig):

$$a_1 = \text{tansig}(W_1 t + b_1), \quad \text{tansig}(z) = \frac{2}{1 + e^{-2z}} - 1 = \tanh z. \quad (2.1)$$

Выходной слой с линейной функцией активации (purelin) линейно комбинирует полученные признаки:

$$y = \text{purelin}(W_2 a_1 + b_2) = \sum_{i=1}^S w_{2,i} \tanh(w_{1,i} t + b_{1,i}) + b_2. \quad (2.2)$$

Таким образом, выход сети — сумма S сдвинутых и масштабированных «ступенек» $\tanh$ плюс смещение; сеть 1– S –1 имеет $3S + 1$ настраиваемых параметров.

2.2 Предобработка и инициализация (как в newff)

Функция newff по умолчанию нормирует входы и цели функцией `mapminmax` в диапазон $[-1, 1]$:

$$\tilde{t} = 2 \frac{t - t_{\min}}{t_{\max} - t_{\min}} - 1, \quad (2.3)$$

что удерживает аргументы $\tanh$ в области без насыщения. Веса скрытого слоя инициализируются по методу Нгуена — Уидроу (`initnw`): для одного входа $|w_{1,i}| = 0,7 S$, а смещения $b_{1,i}$ равномерно распределены по $[-0,7S; 0,7S]$, так что активные области сигмоид с самого начала покрывают весь отрезок входа.

2.3 Алгоритм обратного распространения ошибки

Функция качества — среднеквадратичная ошибка

$$E = \frac{1}{N} \sum_{k=1}^N (\hat{y}_k - y_k)^2. \quad (2.4)$$

Градиенты вычисляются по правилу дифференцирования сложной функции, начиная с выхода:

$$\delta_2 = \frac{2}{N}(\hat{y} - y), \quad \delta_1 = (\delta_2 W_2) \odot (1 - a_1^2), \quad (2.5)$$

$$\frac{\partial E}{\partial W_2} = a_1^\top \delta_2, \quad \frac{\partial E}{\partial b_2} = \sum_k \delta_{2,k}, \quad \frac{\partial E}{\partial W_1} = \tilde{t}^\top \delta_1, \quad \frac{\partial E}{\partial b_1} = \sum_k \delta_{1,k}, \quad (2.6)$$

где использовано $\tanh'(z) = 1 - \tanh^2 z$. Алгоритм `traingd` выполняет пакетный градиентный спуск

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla_{\theta} E(\theta^{(k)}), \quad (2.7)$$

где η — коэффициент обучения (lr). Для квадратичной аппроксимации функции ошибки спуск устойчив только при

$$\eta < \frac{2}{\lambda_{\max}}, \quad (2.8)$$

где $\lambda_{\max}$ — наибольшее собственное число гессиана; скорость сходимости определяется числом обусловленности $\kappa = \lambda_{\max}/\lambda_{\min}$.

Обучение прекращается при выполнении любого из условий: $E \leq \text{goal}$, достигнуто `epochs` эпох, норма градиента меньше `min_grad`. Параметр `show` задаёт период вывода промежуточных результатов.

2.4 Ускоренные алгоритмы обучения

Для сравнения рассмотрены алгоритмы MATLAB:

- `traingdx` — градиентный спуск с моментом ($mc = 0,9$) и адаптивным шагом: при уменьшении ошибки $\eta \leftarrow 1,05 \eta$, при росте ошибки более чем в 1,04 раза шаг отменяется и $\eta \leftarrow 0,7 \eta$;
- `trainlm` — метод Левенберга — Марквардта, использующий якобиан J выходов сети по параметрам:

$$\Delta \theta = -(J^\top J + \mu I)^{-1} J^\top e, \quad (2.9)$$

при малом μ шаг близок к шагу Гаусса — Ньютона, при большом — к короткому градиентному шагу.

2.5 Теорема об универсальной аппроксимации

Согласно теореме Цыбенко (1989) и её обобщению Хорником (1991), для любой непрерывной функции на компакте и любого $\varepsilon > 0$ существует сеть с одним скрытым слоем из конечного числа сигмоидальных нейронов и

линейным выходом, приближающая функцию с точностью ε в равномерной метрике.

3 ХОД РАБОТЫ

3.1 Исходные данные

Вариант 3:

$$f(t) = (t + 1,3) \sin(0,5t + 1), \quad t \in [-7; 7]. \quad (3.1)$$

Сформирован вектор $t = -7 : 0,1 : 7$ (141 точка) и вектор эталонных значений $y = f(t)$, $y \in [-8,11; 4,27]$. Функция имеет три экстремума: максимумы $f(-5,95) = 4,27$ и $f(2,18) = 3,02$ и минимум $f(-1,65) = -0,06$ (рисунок 5.1). Для оценки качества между узлами обучающей сетки использована контрольная сетка с шагом 0,01 (1401 точка), не участвующая в обучении.

3.2 Программная реализация

Методические указания допускают Python или JavaScript вместо MATLAB. Выполнены две реализации без библиотек машинного обучения:

- lab_03.ipynb — Python/NumPy: классы MapMinMax и TwoLayerNet (аналог newff, sim), функция train с алгоритмами traingd, traingdm, traingdx, trainlm и критериями останова MATLAB; протокол обучения выводится каждые show эпох в формате окна MATLAB. Корректность обратного распространения проверена сравнением с численным градиентом: относительная ошибка $4 \cdot 10^{-11}$;
- app/nn_trainer.html — интерактивное веб-приложение на JavaScript и Plotly.js (раздел 3.8).

Все интерактивные графики сохранены в каталоге report/outputs/ в формате HTML (масштабирование, подсказки при наведении, анимация), статические копии — в report/figures/.

3.3 Обучение по заданию с мультипликацией

Сеть с $S = 10$ обучена алгоритмом traingd с параметрами show = 50, lr = 0.05, epochs = 600, goal = 1e-2. Протокол обучения:

```
TRAINGD, Epoch 0/600, MSE 144.86/0.01, Gradient 8.435/1e-05
TRAINGD, Epoch 50/600, MSE 0.79958/0.01, Gradient 0.0934/1e-05
TRAINGD, Epoch 100/600, MSE 0.46114/0.01, Gradient 0.03874/1e-05
TRAINGD, Epoch 200/600, MSE 0.27303/0.01, Gradient 0.02646/1e-05
TRAINGD, Epoch 300/600, MSE 0.16877/0.01, Gradient 0.02034/1e-05
TRAINGD, Epoch 400/600, MSE 0.10635/0.01, Gradient 0.01584/1e-05
TRAINGD, Epoch 500/600, MSE 0.068251/0.01, Gradient 0.01242/1e-05
TRAINGD, Epoch 600/600, MSE 0.044695/0.01, Gradient 0.009792/1e-05
```

Обучение остановлено по лимиту эпох. Итог: $MSE = 0,045$ на обучающей и $0,044$ на контрольной сетке, $R^2 = 0,995$, максимальное отклонение $0,44$ при размахе функции $12,4$. Кривая аппроксимации перерисовывалась каждые 50 эпох синхронно с кривой ошибки (рисунки 5.2 и 5.3): за первые 50 эпох сеть грубо повторяет форму функции (MSE падает со 145 до $0,8$), далее медленно «дотягивает» экстремумы.

3.4 Серия экспериментов с разным числом нейронов

Для $S \in \{1, 2, 5, 10, 20, 30\}$ выполнено по 10 запусков с разной начальной инициализацией при параметрах задания. Результаты приведены в таблице 3.1 и на рисунках 5.4–5.6.

Таблица 3.1 – Серия экспериментов по заданию (traingd, $\eta = 0,05$, 600 эпох, 10 инициализаций)

S	Параметров	MSE медиана	MSE лучшая	MSE худшая	MSE контроль	R^2	Цель
1	4	4.0830	3.2209	4.8133	4.0356	0.5335	0/10
2	7	2.1762	2.1025	2.2680	2.1764	0.7535	0/10
5	16	0.0484	0.0280	0.1664	0.0436	0.9948	0/10
10	31	0.0493	0.0199	0.0840	0.0471	0.9944	0/10
20	61	0.0670	0.0272	0.1045	0.0636	0.9923	0/10
30	91	0.0958	0.0341	0.1208	0.0955	0.9890	0/10

- $S = 1$ и $S = 2$ — явное недообучение: одна сигмоида даёт почти наклонную прямую, две — плавную ступеньку; экстремумы функции пропущены, $R^2 = 0,53$ и $0,75$.
- $S = 5$ и $S = 10$ — форма функции передана полностью, $R^2 \approx 0,995$; остаточная ошибка сосредоточена в экстремумах.
- $S = 20$ и $S = 30$ — медианная MSE выше ($0,067$ и $0,096$), на кривой видна мелкая рябь. Это недоученность, а не переобучение: ошибка на контрольной сетке равна ошибке на обучающей. Инициализация Нгуена — Уидроу делает сигмоиды тем круче, чем больше S ($|w_1| = 0,7S$), и 600 малых шагов не успевают их согласовать.

Цель 10^{-2} за 600 эпох не достигнута ни в одном из 60 запусков: ограничивает не архитектура, а бюджет обучения.

3.5 Доведение обучения до целевой ошибки

Чтобы отделить возможности архитектуры от качества оптимизации, те же сети обучены: алгоритмом traingd без ограничения в 600 эпох (до 20 000), а также алгоритмами traingdx и trainlm при прежнем лимите 600 эпох (таблица 3.2, рисунок 5.7).

Таблица 3.2 – Доведение обучения до целевой ошибки 10^{-2} (медиана по 10 инициализациям)

Алгоритм	S	MSE медиана	MSE контроль	Эпох	Цель
traingd (≤ 20000 эп.)	1	1.5527	1.5578	20000	0/10
traingd (≤ 20000 эп.)	2	0.8646	0.8664	20000	0/10
traingd (≤ 20000 эп.)	5	0.0100	0.0090	3673	10/10
traingd (≤ 20000 эп.)	10	0.0100	0.0090	2360	10/10
traingd (≤ 20000 эп.)	20	0.0100	0.0090	2618	10/10
traingd (≤ 20000 эп.)	30	0.0100	0.0097	3961	10/10
traingdx	1	1.5105	1.5153	600	0/10
traingdx	2	0.7761	0.7779	600	0/10
traingdx	5	0.0100	0.0089	145	10/10
traingdx	10	0.0099	0.0089	117	10/10
traingdx	20	0.0099	0.0089	196	10/10
traingdx	30	0.0100	0.0097	386	10/10
trainlm	1	1.5039	1.5076	25	0/10
trainlm	2	0.6751	0.6770	66	0/10
trainlm	5	0.0071	0.0067	5	10/10
trainlm	10	0.0030	0.0030	3	10/10
trainlm	20	0.0027	0.0026	1	10/10
trainlm	30	9.1e-4	8.6e-4	1	10/10

При $S \geq 5$ цель достигается во всех запусках: traingd требуется 2360–3960 эпох, traingdx — 117–386, trainlm — 1–5 эпох. Сети с $S = 1$ и $S = 2$ не достигают цели ни одним алгоритмом.

Предельная точность при данном S определена обучением trainlm практически до сходимости (goal = 1e-10, до 500 эпох, лучший из 10 запусков), таблица 3.3 и рисунки 5.8–5.10.

Таблица 3.3 – Предельная точность сети с S нейронами (trainlm, лучший из 10 запусков)

S	MSE обучение	MSE контроль	$\max e $
1	1.5e+00	1.5e+00	2.2e+00
2	6.8e-01	6.8e-01	1.6e+00
3	9.3e-05	8.5e-05	4.4e-02
4	8.7e-06	8.3e-06	7.8e-03
5	7.3e-09	6.9e-09	2.9e-04
10	2.5e-09	2.2e-09	2.4e-04
20	9.7e-10	1.0e-09	1.4e-04
30	3.4e-10	4.7e-10	1.7e-04

Минимальное число нейронов, при котором цель 10^{-2} достижима, равно трём — по одному на каждый изгиб функции. Уже при $S = 5$ ошибка снижается до $\sim 10^{-8}$, дальнейшее увеличение S практически ничего не даёт.

Ошибка на контрольной сетке совпадает с ошибкой на обучающей: на точных данных с шагом 0,1 переобучения нет.

3.6 Влияние коэффициента обучения

Для $S = 10$ исследованы значения η из допустимого диапазона $[0,01; 0,5]$ (таблица 3.4, рисунок 5.11).

Таблица 3.4 – Влияние коэффициента обучения (traingd, $S = 10$, 600 эпох, 10 инициализаций)

η	MSE медиана	Цель достигнута	Эпох до цели	Расходимость
0.01	0.2342	0/10	—	0/10
0.05	0.0493	0/10	—	0/10
0.1	0.0262	2/10	477	0/10
0.2	0.0100	8/10	332	0/10
0.25	—	0/10	—	10/10
0.3	—	0/10	—	10/10
0.5	—	0/10	—	10/10

До $\eta = 0,2$ рост шага ускоряет сходимость, при $\eta \geq 0,25$ обучение расходится во всех запусках. Оценка гессиана приближением Гаусса — Ньютона $H \approx \frac{2}{N} J^\top J$ даёт $\lambda_{\max} \approx 10,5$, т.е. критический шаг $2/\lambda_{\max} \approx 0,19$, что согласуется с экспериментом. Число обусловленности $\kappa \sim 10^{11}$ – 10^{12} : поверхность ошибки — узкий «овраг», поэтому допустимый шаг ограничен крутыми направлениями, а вдоль пологих спуск идёт очень медленно. Именно это объясняет тысячи эпох traingd и выигрыш алгоритмов, учитывающих кривизну.

3.7 Недообучение и переобучение

На точной функции с плотной сеткой переобучение не проявляется, поэтому к обучающим значениям добавлен шум $\mathcal{N}(0; 0,7^2)$, а качество оценивается по истинной функции на контрольной сетке. Сети обучены trainlm до минимума ошибки (1000 эпох). Дополнительно для $S = 30$ применён ранний останов: разбиение 70/15/15 % и остановка после 6 последовательных ростов ошибки на валидации (как dividerand и max_fail в MATLAB). Результаты — в таблице 3.5 и на рисунках 5.12–5.13.

Таблица 3.5 – Недообучение и переобучение на зашумлённых данных ($\sigma = 0,7$, trainlm)

S	MSE на шумных точках	MSE относительно $f(t)$
1	1.835	1.512
2	1.041	0.684
3	0.531	0.060
5	0.487	0.066
8	0.409	0.135
10	0.360	0.231
15	0.299	0.266
20	0.249	0.350
30	0.170	4.062
30 + ранний останов	0.277	0.374

Ошибка на обучающих точках монотонно уменьшается с ростом S , а ошибка относительно истинной функции имеет U-образную форму с минимумом при $S = 3$ (0,060). При $S = 30$ ошибка на обучении (0,17) ниже дисперсии шума (0,49) — сеть запоминает шум; кривая проходит через обучающие точки и даёт узкий выброс от -23 до $+32$ около $t \approx 0,1$, ошибка относительно истинной функции — 4,06. Ранний останов уменьшает её до 0,37, т.е. в 11 раз.

3.8 Интерактивное приложение

Приложение `app/nn_trainer.html` (рисунок 5.14) реализует требования раздела 3 методических указаний:

- ползунок числа нейронов скрытого слоя от 1 до 30;
- ручной ввод коэффициента обучения $\eta \in [0,01; 0,5]$ и целевой ошибки $goal \in [10^{-4}; 10^{-1}]$ с проверкой диапазона, а также `epochs`, `show`, выбор алгоритма и начальной инициализации;
- координатная плоскость с точками исходной функции и непрерывной кривой аппроксимации, которая перерисовывается каждые `show` эпох; бледный «след» прошлых положений кривой показывает процесс её изгибания;
- синхронный график ошибки, выстраиваемый точка за точкой, с линией `goal` и отметками перерисовок;
- ползунок «Задержка (мс)» и кнопки «Старт», «Пауза», «Сброс» (также клавиши «Пробел» и «R»);
- дополнительно: выбор любого из 14 вариантов, график вкладов отдельных нейронов, кнопка серийного эксперимента для $S = 1, 2, 5, 10, 20, 30$ с таблицей результатов.

4 РЕЗУЛЬТАТЫ

1. Реализована двухслойная сеть `tansig-purelin` с обучением обратным распространением ошибки; корректность градиентов подтверждена численной проверкой.
2. По параметрам задания (`traingd`, $\eta = 0,05$, 600 эпох) сеть с $S = 10$ аппроксимирует функцию с $MSE = 0,045$ и $R^2 = 0,995$; цель 10^{-2} за 600 эпох не достигается.
3. Точность аппроксимации определяется числом нейронов до $S \approx 5$: $S = 1, 2$ — недообучение (MSE не ниже 1,5 и 0,68 при любом обучении), $S \geq 3$ — цель достижима, $S \geq 5$ — $MSE \sim 10^{-8}$.
4. При фиксированном бюджете 600 эпох большие сети ($S = 20, 30$) обучаются хуже средних из-за более сложной оптимизации.
5. Ускоренные алгоритмы достигают цели на порядки быстрее: `traingdx` — за сотни эпох, `trainlm` — за единицы.
6. Граница устойчивости `traingd` $\eta \approx 0,2$ совпадает с теоретической $2/\lambda_{\max} \approx 0,19$.
7. На зашумлённых данных избыточная сеть ($S = 30$) переобучается; оптимальное $S = 3$, ранний останов снижает ошибку избыточной сети в 11 раз.

5 ИЛЛЮСТРАЦИИ РЕЗУЛЬТАТОВ

Ниже приведены основные визуализации, полученные в ходе выполнения лабораторной работы. Интерактивные версии графиков находятся в каталоге `outputs/`.

Вариант 3: $f(t) = (t + 1.3) \cdot \sin(0.5t + 1)$, $t \in [-7; 7]$

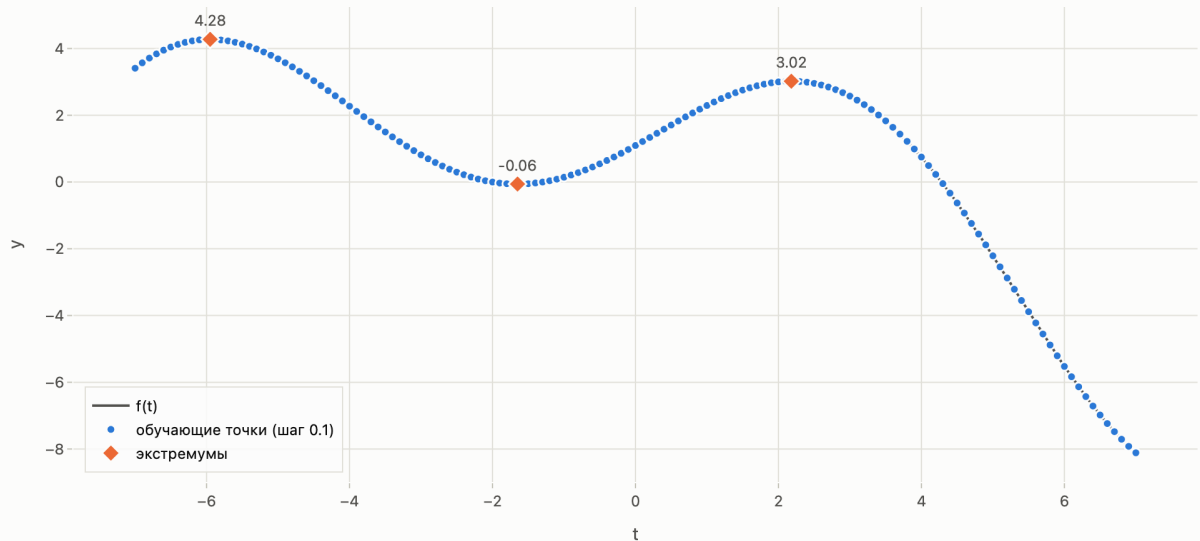


Рисунок 5.1 – Функция варианта 3 и обучающие точки с шагом 0,1.

Мультипликация обучения `traingd`: $S = 10$, $lr = 0.05$, `show = 50`
Эпоха 600 · MSE = 0.0447

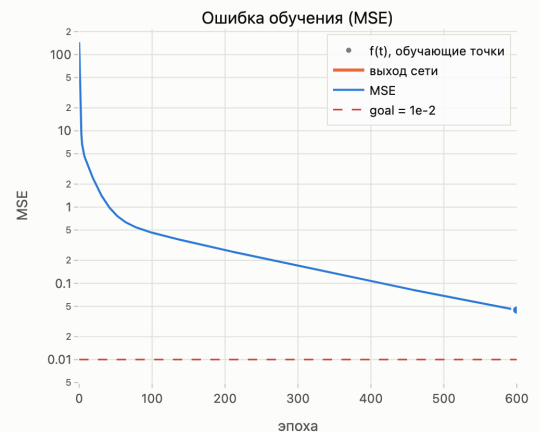
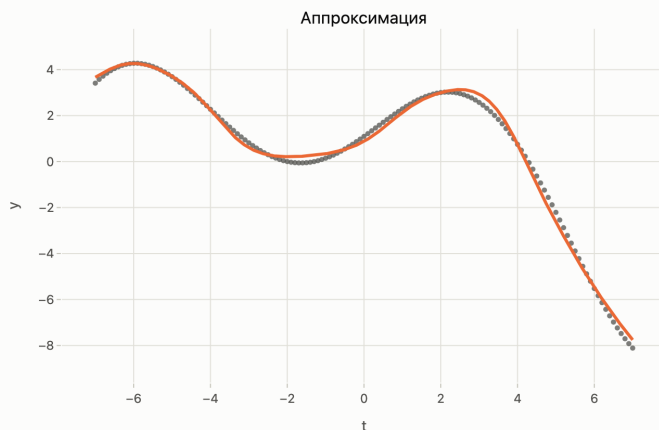


Рисунок 5.2 – Мультипликация обучения `traingd` ($S = 10$): финальный кадр. Интерактивная версия с кнопками «Старт», «Пауза», «Сброс» — `outputs/02_training_animation.html`.

Изгибание кривой аппроксимации в ходе обучения (traingd, $S = 10$)

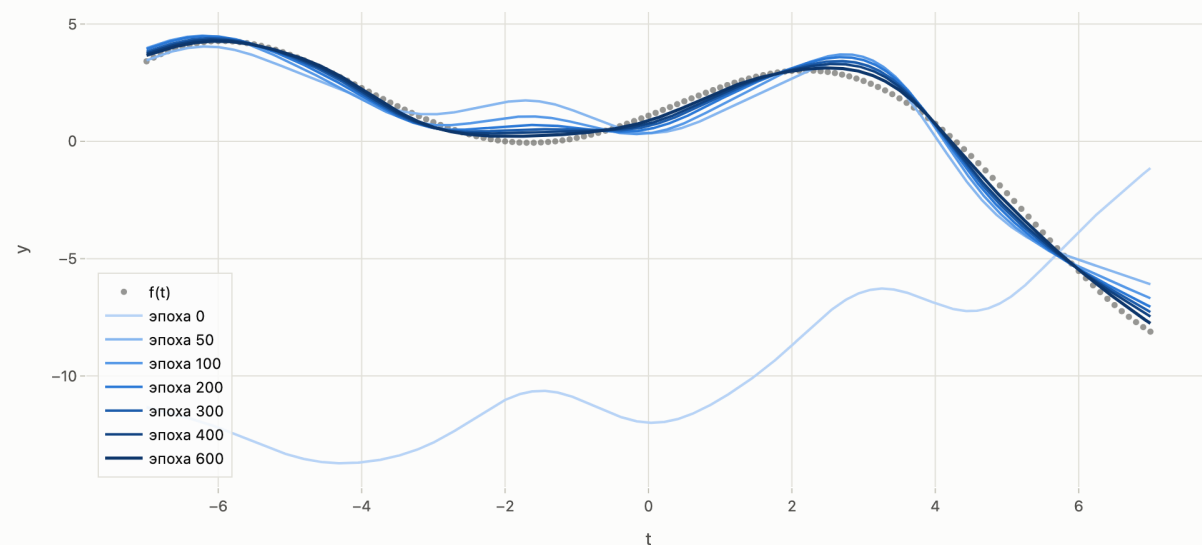


Рисунок 5.3 – Изгибание кривой аппроксимации в ходе обучения traingd ($S = 10$).

traingd, $lr = 0.05$, 600 эпох: аппроксимация при разном числе нейронов

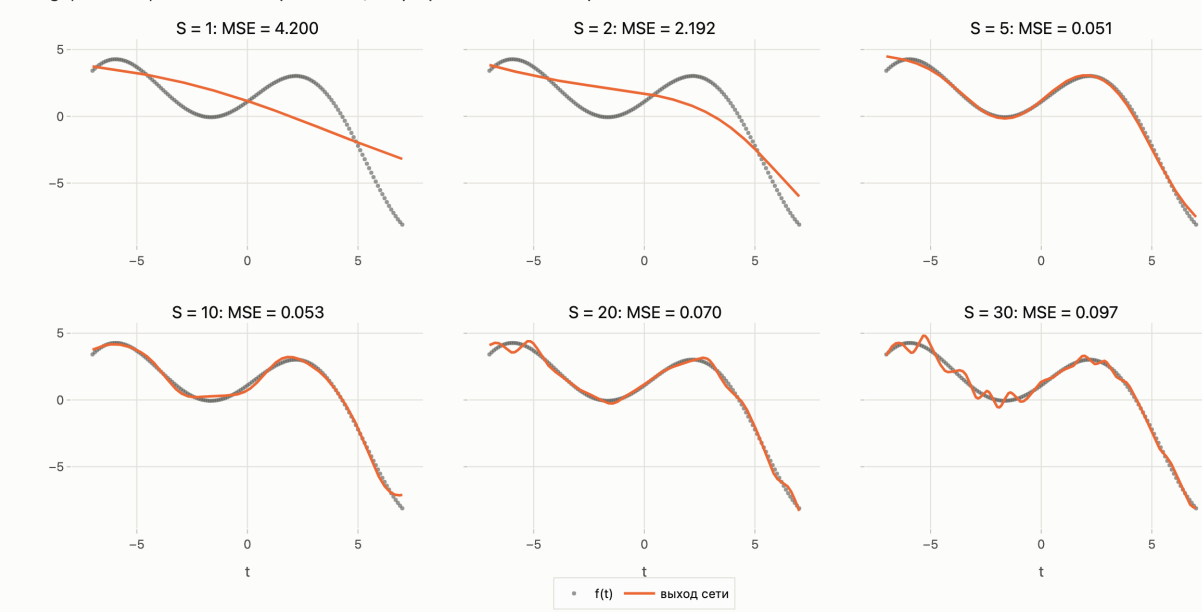


Рисунок 5.4 – Аппроксимация при $S = 1, 2, 5, 10, 20, 30$ (traingd, $\eta = 0,05$, 600 эпох, типичный запуск).

Падение ошибки training при разном S ($lr = 0.05$)

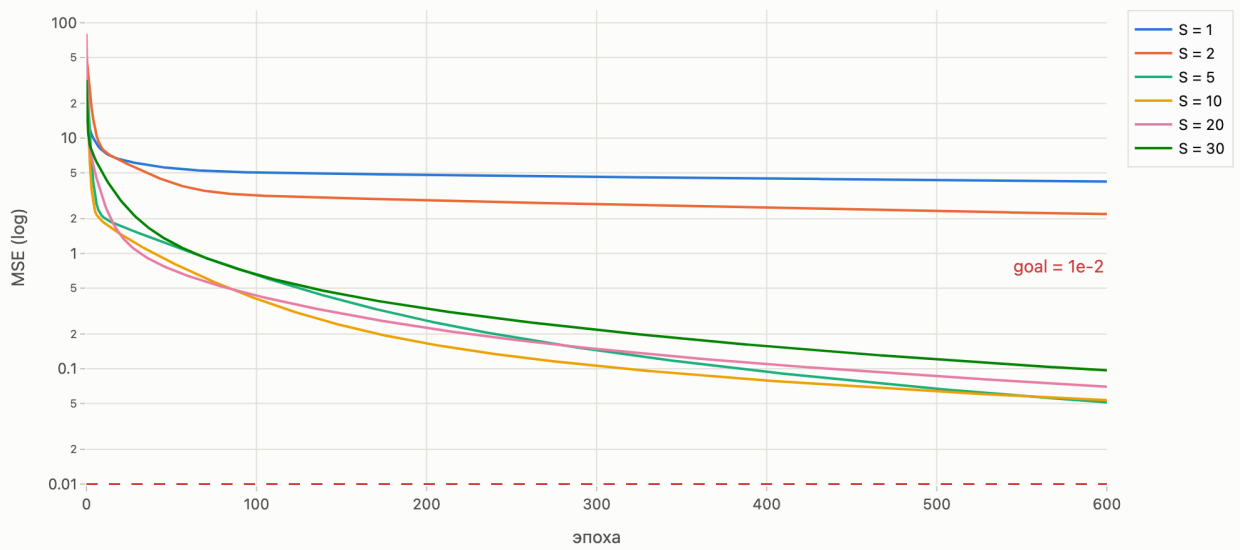


Рисунок 5.5 – Кривые ошибки training при разном числе нейронов.

Итоговая MSE после 600 эпох training: 10 инициализаций на каждое S

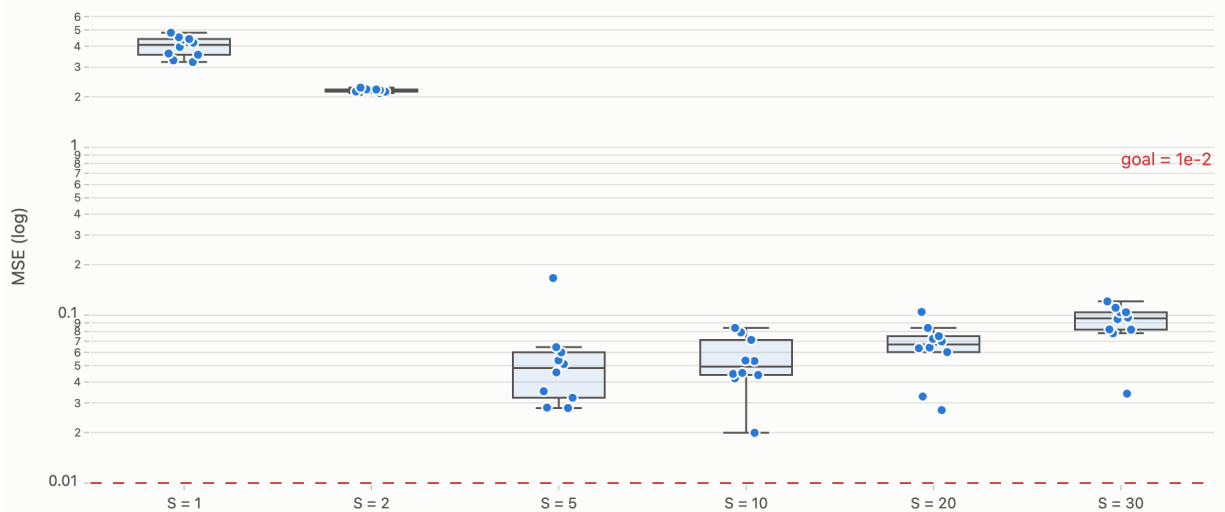


Рисунок 5.6 – Разброс итоговой MSE по 10 инициализациям.

$S = 10$: сколько эпох нужно разным алгоритмам, чтобы достичь goal

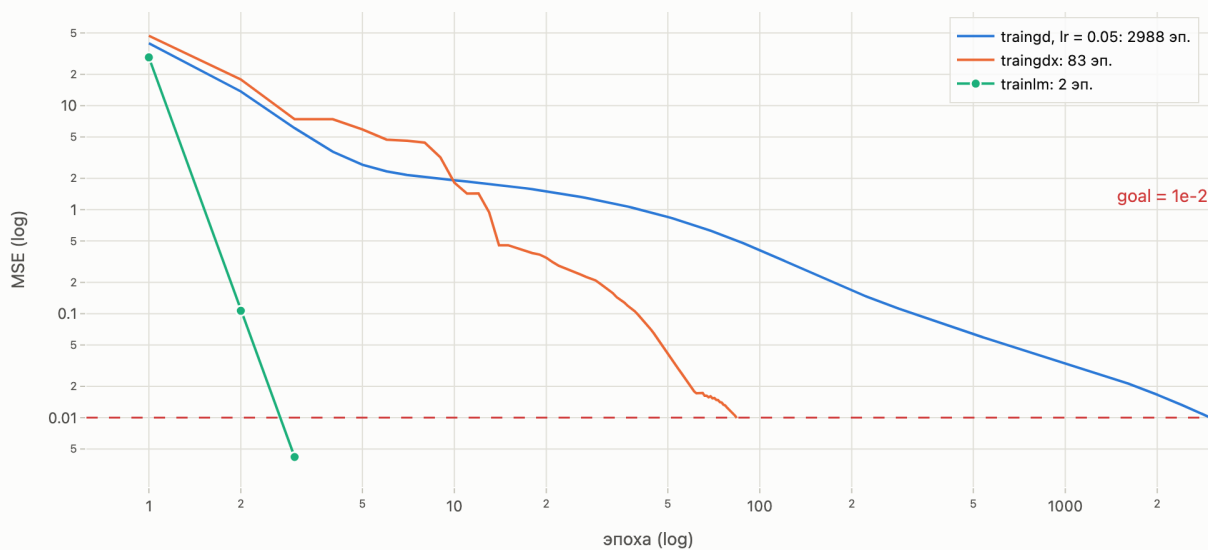


Рисунок 5.7 – Скорость сходимости traingd, trainidx и trainlm при $S = 10$.

Точность аппроксимации в зависимости от числа нейронов скрытого слоя

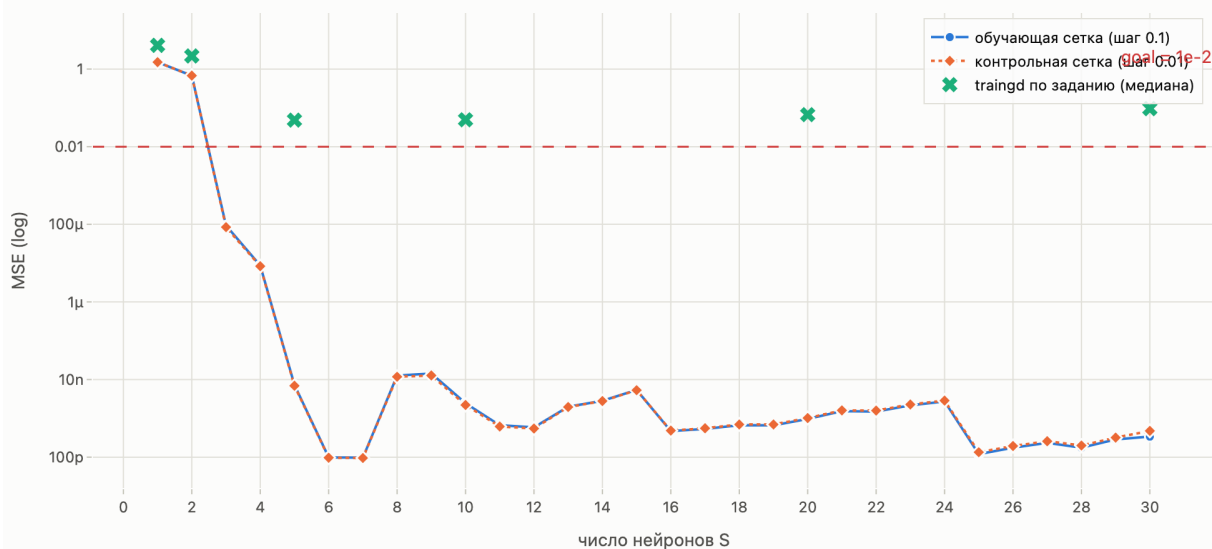


Рисунок 5.8 – Точность аппроксимации в зависимости от числа нейронов скрытого слоя.

Сети, обученные до сходимости (trainlm, лучший из 10 запусков)

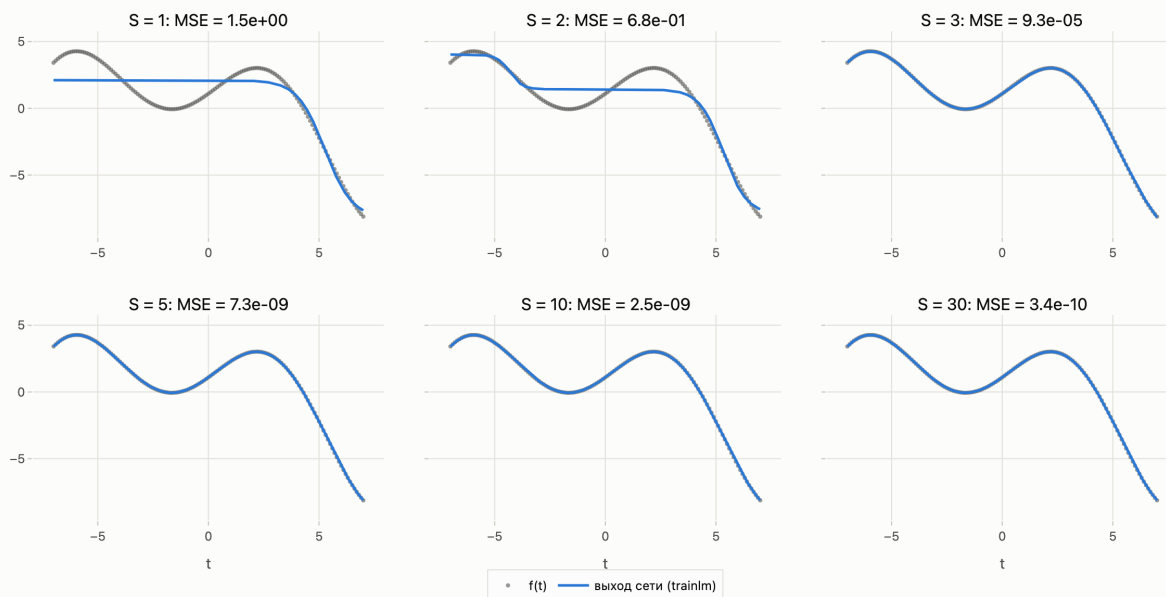


Рисунок 5.9 – Сети, обученные trainlm до сходимости.

Выход сети как сумма сдвинутых и масштабированных гиперболических тангенсов

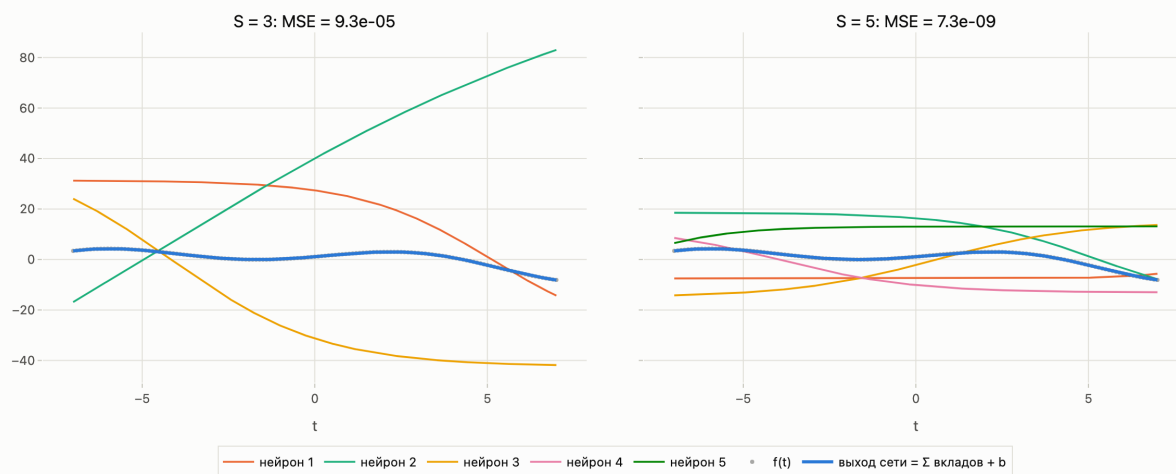


Рисунок 5.10 – Выход сети как сумма вкладов нейронов скрытого слоя ($S = 3$ и $S = 5$).

traingd, $S = 10$: влияние коэффициента обучения на траекторию ошибки

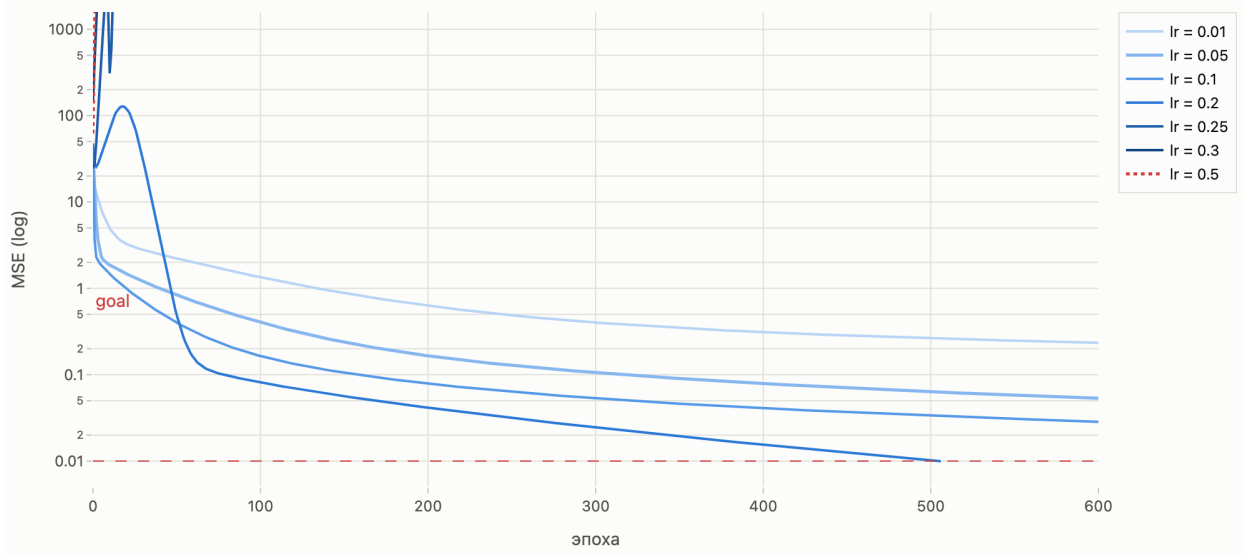


Рисунок 5.11 – Влияние коэффициента обучения на траекторию ошибки traingd ($S = 10$).

Данные с шумом $\sigma = 0.7$: признаки недообучения и переобучения

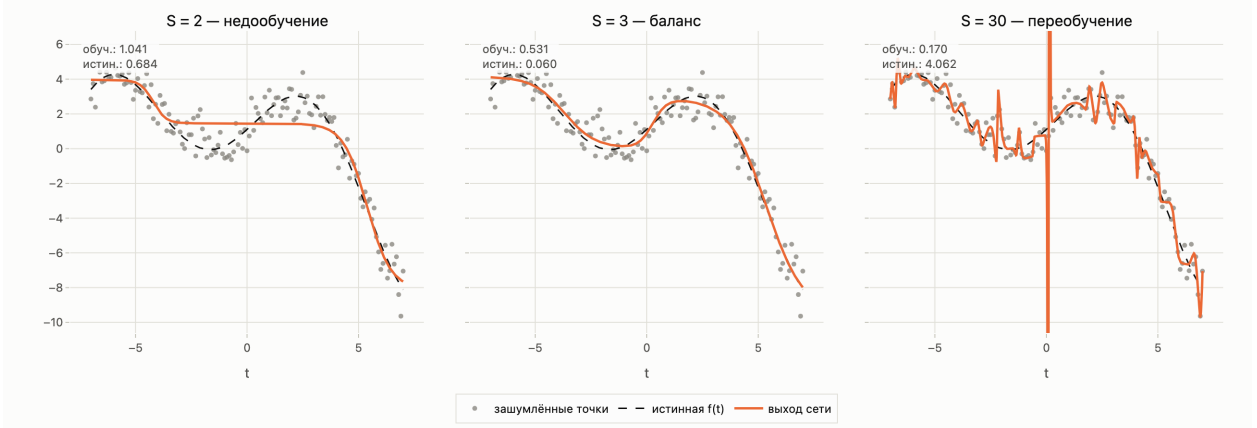


Рисунок 5.12 – Недообучение и переобучение на зашумлённых данных.

Ошибка на обучении падает всегда, ошибка на истинной функции — U-образно

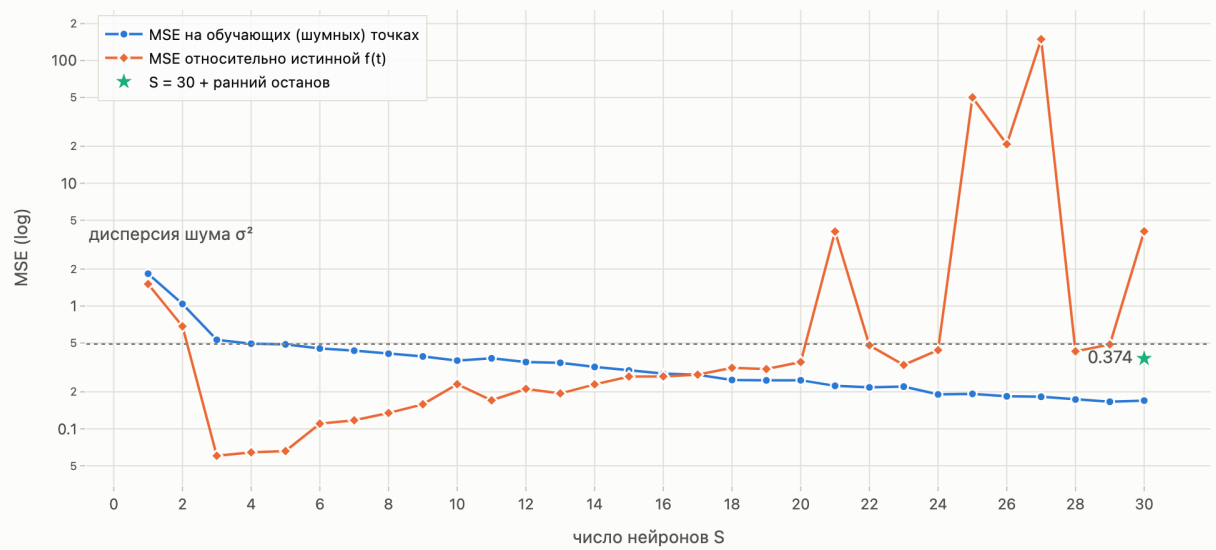


Рисунок 5.13 – Ошибка на обучающих точках и относительно истинной функции в зависимости от S .

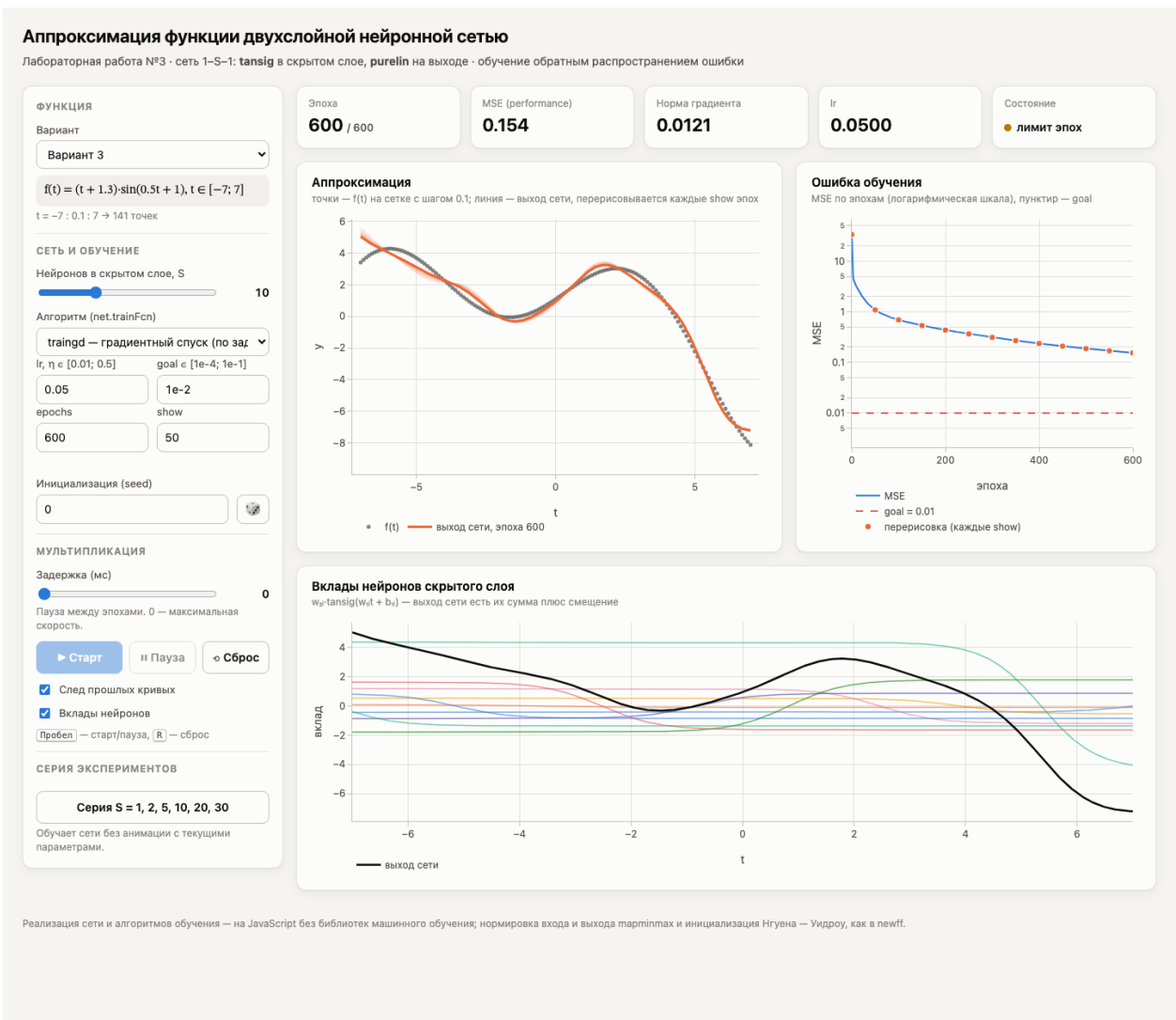


Рисунок 5.14 – Интерактивное приложение `app/nn_trainer.html`.

6 ВЫВОДЫ

В ходе лабораторной работы реализована двухслойная нейронная сеть с сигмоидальным скрытым и линейным выходным слоями, обучаемая методом обратного распространения ошибки, и интерактивное приложение, наглядно показывающее процесс обучения. Экспериментально подтверждена теорема об универсальной аппроксимации: трёх нейронов достаточно, чтобы приблизить функцию варианта 3 с ошибкой 10^{-4} , пяти — с ошибкой 10^{-8} . Показано, что точность, достигаемая на практике, определяется не только числом нейронов, но и алгоритмом обучения: градиентный спуск с постоянным шагом медленно движется по плохо обусловленной поверхности ошибки и за 600 эпох не достигает цели, тогда как методы с адаптивным шагом и учётом кривизны достигают её за сотни и единицы эпох. Слишком малое число нейронов приводит к недообучению, избыточное на зашумлённых данных — к переобучению, которое устраняется уменьшением числа нейронов или ранним остановом по валидационной выборке.

7 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Какова математическая и алгоритмическая роль нелинейной функции активации в скрытом слое при решении задач аппроксимации?

Без нелинейности сеть вырождается в линейную модель: $W_2(W_1t + b_1) + b_2 = Wt + b$, поскольку композиция линейных отображений линейна, и при любом числе нейронов получается прямая. Функция `tansig` превращает каждый нейрон в гладкую «ступеньку» $\tanh(wt + b)$ с настраиваемыми положением $(-b/w)$, крутизной (w) и высотой (w_2) ; выходной слой складывает эти ступеньки, и из их суммы можно собрать кривую любой формы (рисунок 5.10). Алгоритмически важно, что `tansig` дифференцируема, а её производная выражается через уже вычисленное значение: $\tanh'(z) = 1 - \tanh^2 z$, что делает обратное распространение дешёвым. Ограниченность и симметричность относительно нуля улучшают обусловленность обучения.

2. Опишите физический и математический смысл алгоритма обратного распространения ошибки (`traingd`). Как коэффициент обучения (`lr`) влияет на сходимость траектории?

Математически `traingd` — градиентный спуск $\theta \leftarrow \theta - \eta \nabla E(\theta)$ по функции MSE. Обратное распространение — эффективный способ вычислить градиент по правилу дифференцирования сложной функции: ошибка выхода $\delta_2 = \frac{2}{N}(\hat{y} - y)$ передаётся назад через веса W_2 и умножается на производную активации, $\delta_1 = (\delta_2 W_2) \odot (1 - a_1^2)$; градиент по весу равен произведению сигнала ошибки на вход этого веса. Физически это движение по поверхности ошибки в направлении наискорейшего спуска, при котором каждый вес получает поправку, пропорциональную своему вкладу в ошибку.

Коэффициент `lr` задаёт длину шага. Малый шаг (0,01) даёт устойчивую, но очень медленную траекторию (MSE 0,23 за 600 эпох). Увеличение шага ускоряет спуск ($\eta = 0,2$ — цель примерно за 330 эпох), но при $\eta > 2/\lambda_{\max}$ (здесь $\approx 0,19$) траектория перескакивает дно «оврага», колеблется с растущей амплитудой и расходится ($\eta \geq 0,25$). Поскольку поверхность ошибки сильно вытянута ($\kappa \sim 10^{11}$), один постоянный шаг не подходит одновременно для крутых и пологих направлений — отсюда преимущество `traingdx` и `trainlm`.

3. Сформулируйте суть теоремы об универсальной аппроксимации (теорема Цыбенко / Колмогорова). Что она накладывает на количество слоёв сети?

Теорема Цыбенко (1989), обобщённая Хорником (1991): для любой непрерывной функции f на компакте $K \subset \mathbb{R}^n$ и любого $\varepsilon > 0$ существует сеть с *одним* скрытым слоем из конечного числа S сигмоидальных нейронов и линейным выходом, такая что $\max_{x \in K} |f(x) - \hat{f}(x)| < \varepsilon$. Теорема Колмогорова — Арнольда утверждает, что любую непрерывную функцию n переменных можно точно представить суперпозицией непрерывных функций одной переменной и операции сложения. Следствие для архитектуры: двух слоёв (нелинейный скрытый и линейный выходной) *достаточно*,

дополнительные слои для самой возможности аппроксимации не нужны. Однако теорема не указывает требуемое число нейронов, не гарантирует, что веса будут найдены градиентным спуском, и ничего не говорит о поведении сети вне обучающих точек. Эксперимент это иллюстрирует: при $S \geq 3$ точность 10^{-2} достижима, но `traingd` за 600 эпох её не находит.

4. Каковы явные визуальные признаки недообучения и переобучения (overfitting) сети, наблюдаемые на графике аппроксимации при манипуляциях с размером скрытого слоя?

Недообучение ($S = 1, 2$ или слишком короткое обучение): кривая сети слишком «жёсткая» — гладкая, с меньшим числом изгибов, чем у функции; экстремумы пропущены ($S = 1$ — почти наклонная прямая, $S = 2$ — ступенька); ошибка систематическая, большая и одинаковая на обучающих и контрольных точках.

Переобучение ($S = 30$ на зашумлённых данных): кривая проходит почти через каждую обучающую точку, повторяя шум; между точками возникают осцилляции и резкие выбросы (у нас — от -23 до $+32$ около $t \approx 0,1$ при размахе функции около 12); ошибка на обучающих точках ниже уровня шума, а относительно истинной функции многократно выше. Главный признак — большой разрыв между ошибками на обучающей и контрольной выборках.

От переобучения следует отличать мелкую рябь у сетей с $S = 20, 30$ после 600 эпох `traingd`: ошибка на контрольной сетке равна ошибке на обучающей, это недоученность, и она исчезает при продолжении обучения.

5. Объясните назначение параметров `net.trainParam.show` и `net.trainParam.goal`.

`show = 50` — период вывода промежуточных результатов: каждые 50 эпох печатается строка протокола (номер эпохи, MSE, норма градиента) и обновляется график; на сам процесс обучения параметр не влияет. В данной работе по нему выполняется перерисовка кривой аппроксимации для мультипликации. `goal = 1e-2` — целевое значение функции качества: обучение прекращается досрочно, как только $E \leq \text{goal}$. Это критерий достаточной точности, экономящий эпохи; наряду с ним действуют лимит `epochs` и порог `min_grad`. Слишком малое значение `goal` на зашумлённых данных толкает сеть к переобучению: ошибки ниже уровня шума можно добиться, только подгоняясь под шум.

8 КОНТРОЛЬНЫЕ ВОПРОСЫ ДЛЯ MOODLE

Ниже приведены 30 вопросов в форме, удобной для визуальной проверки. Исходный банк вопросов для загрузки в Moodle сохранен отдельно в файле `questions.gift.txt`.

1. Какую архитектуру имеет сеть, используемая в лабораторной работе для аппроксимации функции $y = f(t)$?
- ☐ Один вход, скрытый слой из S нейронов с `tansig` и один выходной нейрон с `purelin`
 - ☐ Один вход, два скрытых слоя с `purelin` и выход с `tansig`
 - ☐ Сеть Кохонена с конкурентным слоем
 - ☐ Однослойный персептрон с пороговой функцией активации

Правильный ответ: Один вход, скрытый слой из S нейронов с `tansig` и один выходной нейрон с `purelin`.

2. Какая функция активации используется в скрытом слое сети?

- ☐ `tansig` — гиперболический тангенс
- ☐ `purelin` — линейная функция
- ☐ `hardlim` — пороговая функция
- ☐ `compet` — конкурентная функция

Правильный ответ: `tansig` — гиперболический тангенс.

3. Почему в выходном слое сети для аппроксимации используется линейная функция `purelin`?

- ☐ Потому что выход должен принимать любые вещественные значения, а не только значения из $(-1, 1)$
- ☐ Потому что линейная функция ускоряет вычисление градиента скрытого слоя
- ☐ Потому что с `purelin` сеть не может переобучиться
- ☐ Потому что `tansig` нельзя использовать более одного раза в сети

Правильный ответ: Потому что выход должен принимать любые вещественные значения, а не только значения из $(-1, 1)$.

4. Что произойдет, если в скрытом слое заменить `tansig` на линейную функцию?

- ☐ Сеть станет эквивалентна линейной модели $y = w \cdot t + b$ независимо от числа нейронов
- ☐ Сеть станет точнее аппроксимировать нелинейные функции
- ☐ Обучение перестанет сходиться при любом коэффициенте обучения
- ☐ Число параметров сети уменьшится вдвое

Правильный ответ: Сеть станет эквивалентна линейной модели $y = w \cdot t + b$ независимо от числа нейронов.

5. Сколько настраиваемых параметров имеет сеть 1– S –1 (один вход, S скрытых нейронов, один выход)?

- ☐ $3S + 1$
- ☐ $2S$
- ☐ $S + 1$
- ☐ S в квадрате

Правильный ответ: $3S + 1$.

6. Какая формула задаёт производную функции $\tanh$, используемую в обратном распространении ошибки?

- ☐ $1 - \tanh^2(z)$
- ☐ $\tanh(z) \cdot (1 - \tanh(z))$
- ☐ $1/(1 + e^{-z})$
- ☐ $\tanh(z) - 1$

Правильный ответ: $1 - \tanh^2(z)$.

7. Что делает алгоритм обучения `traingd`?

- ☐ Выполняет пакетный градиентный спуск: изменяет веса на $-lr \cdot \nabla E$ после каждой эпохи
- ☐ Использует метод Левенберга — Марквардта с адаптивным параметром μ
- ☐ Случайно перебирает веса и сохраняет лучшие
- ☐ Обучает сеть без вычисления градиентов по правилу Хебба

Правильный ответ: Выполняет пакетный градиентный спуск: изменяет веса на $-lr \cdot \nabla E$ после каждой эпохи.

8. На каком математическом правиле основан алгоритм обратного распространения ошибки?

- ☐ На правиле дифференцирования сложной функции (цепном правиле)
- ☐ На формуле Байеса
- ☐ На методе наименьших модулей
- ☐ На теореме о среднем значении

Правильный ответ: На правиле дифференцирования сложной функции (цепном правиле).

9. Какая функция ошибки (performance) минимизируется при обучении сети в лабораторной работе?

- ☐ Среднеквадратичная ошибка MSE
- ☐ Перекрёстная энтропия
- ☐ Максимальная абсолютная ошибка
- ☐ Коэффициент детерминации R^2

Правильный ответ: Среднеквадратичная ошибка MSE.

10. Что задаёт параметр `net.trainParam.show = 50`?

- ☐ Период вывода промежуточных результатов и обновления графиков — каждые 50 эпох
- ☐ Количество нейронов скрытого слоя
- ☐ Максимальное число эпох обучения
- ☐ Размер мини-пакета при обучении

Правильный ответ: Период вывода промежуточных результатов и обновления графиков — каждые 50 эпох.

11. Что означает параметр `net.trainParam.goal = 1e-2`?

- ☐ Обучение досрочно прекращается, когда ошибка MSE становится не больше 0.01
- ☐ Коэффициент обучения равен 0.01
- ☐ Веса инициализируются в диапазоне ± 0.01
- ☐ Каждая эпоха длится 0.01 секунды

Правильный ответ: Обучение досрочно прекращается, когда ошибка MSE становится не больше 0.01.

12. Что задаёт параметр `net.trainParam.epochs = 600`?

- ☐ Максимальное число эпох (итераций) обучения
- ☐ Число обучающих точек
- ☐ Число нейронов скрытого слоя
- ☐ Количество перезапусков обучения

Правильный ответ: Максимальное число эпох (итераций) обучения.

13. Как влияет слишком большой коэффициент обучения lr на градиентный спуск?

- ☐ Траектория перескакивает через минимум, колеблется и может разойтись
- ☐ Обучение становится устойчивее, но медленнее
- ☐ Сеть всегда быстрее достигает глобального минимума
- ☐ Ничего не меняется, так как lr влияет только на вывод графиков

Правильный ответ: Траектория перескакивает через минимум, колеблется и может разойтись.

14. Как влияет слишком малый коэффициент обучения lr на градиентный спуск?

- ☐ Обучение устойчиво, но очень медленно, и за отведённые эпохи цель может быть не достигнута
- ☐ Обучение расходится
- ☐ Сеть переобучается уже на первой эпохе
- ☐ Градиенты перестают вычисляться

Правильный ответ: Обучение устойчиво, но очень медленно, и за отведённые эпохи цель может быть не достигнута.

15. Какое условие устойчивости градиентного спуска для квадратичной функции ошибки с наибольшим собственным числом гессиана $\lambda_{\max}$?

- ☐ $lr < 2 / \lambda_{\max}$
- ☐ $lr > \lambda_{\max}$
- ☐ $lr = \lambda_{\max} / 2$
- ☐ $lr < 1 / N$, где N — число точек

Правильный ответ: $lr < 2 / \lambda_{\max}$.

16. Что утверждает теорема об универсальной аппроксимации (Цыбенко, Хорник)?

- ☐ Сеть с одним скрытым слоем с сигмоидальной активацией и достаточным числом нейронов приближает любую непрерывную функцию на компакте с любой точностью
- ☐ Любая сеть с двумя слоями обучается градиентным спуском за конечное число эпох
- ☐ Для аппроксимации функции n переменных нужно не менее n скрытых слоёв
- ☐ Ошибка обучения всегда равна ошибке на новых данных

Правильный ответ: Сеть с одним скрытым слоем с сигмоидальной активацией и достаточным числом нейронов приближает любую непрерывную функцию на компакте с любой точностью.

17. Чего НЕ гарантирует теорема об универсальной аппроксимации?

- ☐ Того, что нужные веса будут найдены алгоритмом обучения, и того, сколько нейронов понадобится
- ☐ Существования сети с одним скрытым слоем, приближающей функцию
- ☐ Возможности сколь угодно точного приближения непрерывной функции
- ☐ Применимости к сигмоидальным функциям активации

Правильный ответ: Того, что нужные веса будут найдены алгоритмом обучения, и того, сколько нейронов понадобится.

18. Какой вид имеет кривая аппроксимации при недообучении (например, $S = 1$)?

- ☐ Слишком гладкая и «жёсткая», пропускает экстремумы функции; ошибка велика и на обучающих, и на новых точках
- ☐ Проходит через каждую обучающую точку и сильно колеблется между ними
- ☐ Совпадает с функцией на обучающих точках, но расходится только вне отрезка
- ☐ Всегда является горизонтальной прямой $y = 0$

Правильный ответ: Слишком гладкая и «жёсткая», пропускает экстремумы функции; ошибка велика и на обучающих, и на новых точках.

19. Какой визуальный признак переобучения сети?

- ☐ Кривая повторяет шум обучающих точек и осциллирует между ними; ошибка на обучении мала, на контрольных точках велика
- ☐ Кривая проходит ниже всех обучающих точек
- ☐ Кривая монотонна на всём отрезке
- ☐ Ошибка одинаково велика на обучающих и контрольных точках

Правильный ответ: Кривая повторяет шум обучающих точек и осциллирует между ними; ошибка на обучении мала, на контрольных точках велика.

20. Как в общем случае меняются ошибка на обучающих данных и ошибка на новых данных при росте числа нейронов S на зашумлённой выборке?

- ☐ Ошибка на обучении уменьшается, а ошибка на новых данных сначала падает, затем растёт (U-образно)
- ☐ Обе ошибки монотонно растут
- ☐ Обе ошибки монотонно убывают до нуля
- ☐ Ошибка на обучении растёт, а на новых данных убывает

Правильный ответ: Ошибка на обучении уменьшается, а ошибка на новых данных сначала падает, затем растёт (U-образно).

21. Как узнать, что мелкая «рябь» на кривой после короткого обучения большой сети — это недоученность, а не переобучение?

- ☐ Ошибка на контрольных точках между узлами практически равна ошибке на обучающих точках и уменьшается при продолжении обучения
- ☐ Рябь всегда означает переобучение
- ☐ Нужно уменьшить число обучающих точек
- ☐ Достаточно посмотреть на значение параметра `show`

Правильный ответ: Ошибка на контрольных точках между узлами практически равна ошибке на обучающих точках и уменьшается при продолжении обучения.

22. Зачем в `newff` входы и цели по умолчанию нормируются функцией `mapminmax`?

- ☐ Чтобы привести данные к диапазону $[-1, 1]$, где $\tanh$ не насыщается и градиенты хорошо обусловлены
- ☐ Чтобы уменьшить число эпох до одной
- ☐ Чтобы сеть могла работать без скрытого слоя
- ☐ Чтобы исключить из выборки выбросы

Правильный ответ: Чтобы привести данные к диапазону $[-1, 1]$, где $\tanh$ не насыщается и градиенты хорошо обусловлены.

23. В чём идея инициализации Нгуена — Уидроу (initnw)?

- ☐ Распределить активные области сигмоид скрытых нейронов равномерно по диапазону входа
- ☐ Инициализировать все веса нулями
- ☐ Задать всем нейронам одинаковые веса и смещения
- ☐ Выбрать веса так, чтобы сеть сразу была линейной

Правильный ответ: Распределить активные области сигмоид скрытых нейронов равномерно по диапазону входа.

24. Почему нельзя инициализировать все веса скрытого слоя одинаковыми значениями?

- ☐ Все нейроны получают одинаковые градиенты и останутся одинаковыми — сеть фактически будет иметь один нейрон
- ☐ Потому что сеть сразу переобучится
- ☐ Потому что MSE станет отрицательной
- ☐ Потому что тогда нельзя вычислить выход purelin

Правильный ответ: Все нейроны получают одинаковые градиенты и останутся одинаковыми — сеть фактически будет иметь один нейрон.

25. Чем алгоритм `trainidx` отличается от `traingd`?

- ☐ Использует момент и адаптивно меняет коэффициент обучения в зависимости от изменения ошибки
- ☐ Использует матрицу Якоби и решает систему линейных уравнений
- ☐ Обучает только выходной слой
- ☐ Не использует градиент ошибки

Правильный ответ: Использует момент и адаптивно меняет коэффициент обучения в зависимости от изменения ошибки.

26. Почему метод Левенберга — Марквардта (`trainlm`) сходится за единицы эпох там, где `traingd` требует тысячи?

- ☐ Он учитывает кривизну поверхности ошибки через приближение гессиана $J^T J$ и делает шаг, близкий к ньютоновскому
- ☐ Он использует больший фиксированный коэффициент обучения
- ☐ Он пропускает вычисление ошибки на каждой эпохе
- ☐ Он уменьшает число параметров сети

Правильный ответ: Он учитывает кривизну поверхности ошибки через приближение гессиана $J^T J$ и делает шаг, близкий к ньютоновскому.

27. Что показывает мультипликация процесса обучения — перерисовка кривой аппроксимации каждые `show` эпох?

- ☐ Как кривая сети постепенно «изгибается» и подстраивается под форму эталонной функции по мере уменьшения ошибки
- ☐ Как меняется число нейронов в скрытом слое
- ☐ Как меняется исходная функция $f(t)$
- ☐ Как сеть выбирает шаг дискретизации t

Правильный ответ: Как кривая сети постепенно «изгибается» и подстраивается под форму эталонной функции по мере уменьшения ошибки.

28. Как формируется выход двухслойной сети с $\tanh$ и purelin ?

- ☐ Как взвешенная сумма сдвинутых и масштабированных гиперболических тангенсов плюс смещение
- ☐ Как произведение выходов всех скрытых нейронов
- ☐ Как максимум из выходов скрытых нейронов
- ☐ Как выход одного случайно выбранного нейрона

Правильный ответ: Как взвешенная сумма сдвинутых и масштабированных гиперболических тангенсов плюс смещение.

29. Что помогает бороться с переобучением сети на зашумлённых данных?

- ☐ Ранний останов по валидационной выборке, уменьшение числа нейронов или регуляризация
- ☐ Увеличение числа нейронов до максимума
- ☐ Уменьшение целевой ошибки goal до нуля
- ☐ Отказ от контрольной выборки

Правильный ответ: Ранний останов по валидационной выборке, уменьшение числа нейронов или регуляризация.

30. Зачем при исследовании влияния числа нейронов повторять обучение с разными начальными весами?

- ☐ Потому что результат градиентного спуска зависит от инициализации, и один запуск может оказаться случайно удачным или неудачным
- ☐ Потому что при одном запуске сеть не может вычислить градиент
- ☐ Потому что каждое новое обучение увеличивает число нейронов
- ☐ Потому что так требует функция trainmax

Правильный ответ: Потому что результат градиентного спуска зависит от инициализации, и один запуск может оказаться случайно удачным или неудачным.